

Sistem distribuit de monitorizare si control

Codreanu Dan

Capitolul 1 Descriere

Sistemul propus reprezinta o solutie integrata de monitorizare si control ambiental de tip Smart Home, bazata pe arhitectura IoT (Internet of Things). Scopul principal este automatizarea climatizarii si a iluminatului intr-un spatiu interior prin comunicarea intre doua noduri inteligente independente (ESP32) care fac schimb de date in timp real prin protocolul MQTT. Sistemul nu doar monitorizeaza parametrii de mediu (temperatura, calitate aer, lumina), ci si actioneaza asupra unor sarcini de putere (motor DC, LED) intr-un mod eficient, protejand componentele hardware prin logica de control avansata.

1.1 Detalii Tehnice: Nodul 1 (Control Putere si Executie)

Acest nod este responsabil pentru gestionarea elementelor mecanice si monitorizarea consumului de energie.

- **Actionare Motor:** Utilizeaza un tranzistor Darlington **TIP142** pentru a controla un motor DC prin PWM (1 kHz).
- **Monitorizare Energie:** Senzorul **MAX471** masoara in timp real curentul si tensiunea consumata de motor, oferind date critice despre sarcina sistemului.
- **Senzor LDR Digital:** Monitorizeaza pragul de luminozitate ambientala.
- **Interfata Locala:** Un ecran **LCD** afiseaza statusul motorului, consumul de curent si starea conexiunii WiFi/MQTT.

1.2 Detalii Tehnice: Nodul 2 (Monitorizare Mediu si Feedback)

Acest nod functioneaza ca unitate de achizitie date si indicator vizual.

- **Senzor BME680:** Senzor de inalta precizie care furnizeaza date despre temperatura, umiditate, presiune atmosferica si calitatea aerului (IAQ - Indoor Air Quality).
- **Sistem Iluminat:** Un LED de controlat prin transistor prin PWM, a carui stare depinde de datele primite de la senzorul LDR de pe Nodul 1.
- **Interfata Locala:** Ecran **LCD** dedicat pentru afisarea telemetriei de mediu primite de la BME680 si starea conexiunii WiFi/MQTT.

1.3 Logica de Control si Comunicare

Interactiunea dintre cele doua noduri este mediata de un broker MQTT si un dashboard Node-RED:

1. **Control Automat Motor (Histerezis):** Nodul 1 primeste temperatura de la Nodul 2 si ajusteaza viteza motorului in functie de diferenta fata de pragul setat.
2. **Control Automat LED :** Cand senzorul LDR (Nod 1) detecteaza intuneric, se trimite o comanda MQTT catre Nodul 2 pentru a aprinde LED-ul local.
3. **Supraveghere:** Node-RED serveste drept interfata centrala de comanda, unde utilizatorul poate trece sistemul din modul "Auto" in "Manual" (Overdrive).

Capitolul 2 Hardware

Acest capitol descrie componentele electronice utilizate si modul in care acestea au fost integrate pentru a realiza cele doua noduri de control si monitorizare.

2.1. Unități de Procesare și Comunicare

Sistemul utilizează două unități **ESP32**, configurate să comunice prin protocolul MQTT. Alegerea acestora a fost motivată de prezența modulelor WiFi integrate și a pinilor hardware dedicați pentru PWM și ADC (conversie analog-digitală, Fig. 2.1).

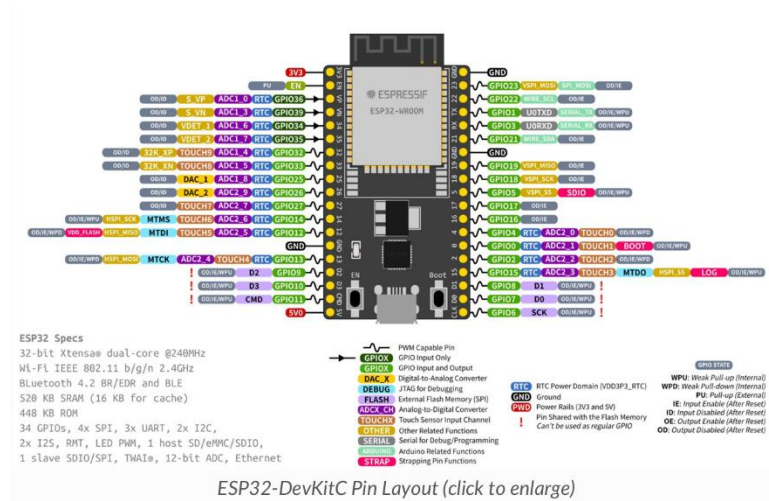


Fig 2.1 : ESP 32

2.2. Senzori (Achiziție Date)

Sistemul colectează date din mediul exterior prin trei senzori principali:

- **BME680:** Un senzor complex de mediu care măsoară temperatura, umiditatea, presiunea atmosferică și rezistența gazului (pentru calitatea aerului Tabel 2.1 si Fig. 2.2).

Parametru	Specificatie
Interfata de comunicare	I2C (pana la 3.4 MHz) si SPI
Gama masurare Temperatura	-40°C ... +85°C
Rezolutie Temperatura	0.01 °C
Toleranta Temperatura	±0.5 °C (la 25°C)
Gama masurare Umiditate	0% ... 100% r.H.
Toleranta Umiditate	±3% r.H.
Gama Presiune Barometrica	300 ... 1100 hPa
Calitate Aer (IAQ)	0 ... 500 (VOC - Compusi Organici Volatili)

Tabel 2.1: Specificatii BME 680



Fig. 2.2: Senzor BME 680

- **LDR (Light Dependent Resistor):** Modul digital care monitorizează intensitatea luminii ambientale pentru controlul automat al iluminatului (Fig. 2.2.)

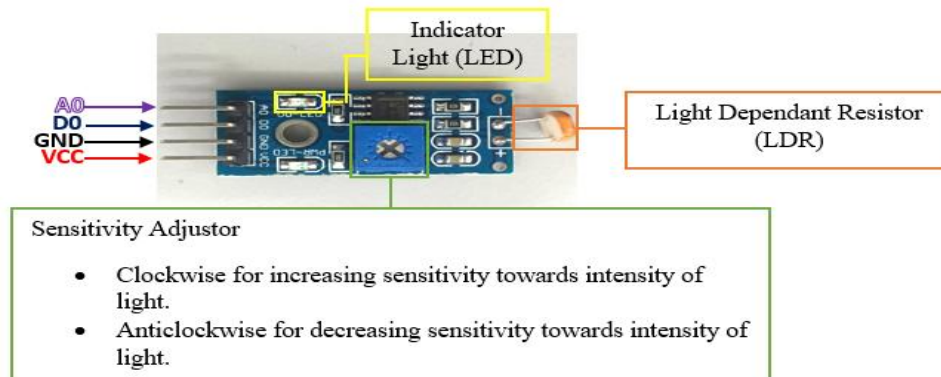


Fig. 2.3: Modul sensor LDR

- **MAX471:** Sensor specializat pentru măsurarea tensiunii și a curentului (până la 3A). Acesta oferă feedback vital despre consumul motorului și eficiența tranzistorului de putere (Tabel 2.2 și Fig. 2.3).

Parametru	Specificatie
Interfata de comunicare	Analogica (Tensiune proportionala cu I și V)
Gama masurare Curent	0A ... 3A (Sensibilitate 1V / 1A)
Gama masurare Tensiune	3V ... 36V (Divizor intern 1:5)
Gama masurare Tensiune	±2%
Rezoluție (pe ESP32)	12-bit ADC (aprox. 0.8mA per unitate)
Funcție principală	Monitorizare consum motor și detecție Back EMF

Tabel 2.2: Specificatii MAX 471

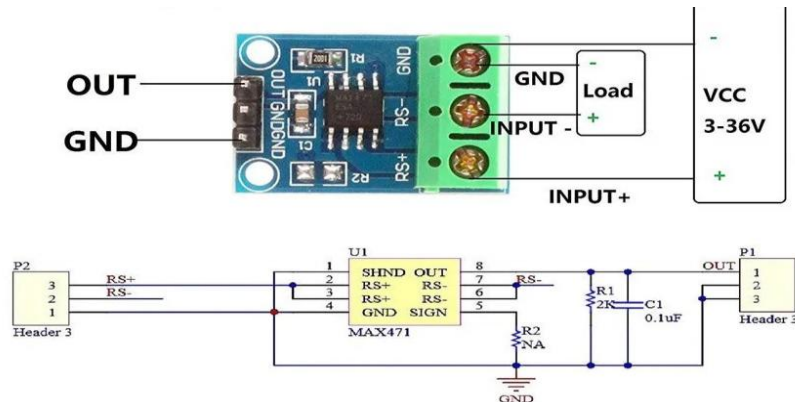


Fig. 2.3: Senzor MAX 471 cu schema de cablare

2.3. Actuatori și Circuite de Comandă

Elementele de execuție sunt controlate prin tranzistori pentru a izola microcontrolerul de sarcinile de putere:

1. Sistemul de Răcire (Motor DC):

- Utilizează un tranzistor Darlington **TIP142** (Fig. 2.4).
- Motorul are o forță nominală de **10 g*cm** (aprox. **0.139 oz*in**).
- Include o **diodă Flyback** pentru protecție împotriva tensiunii electromotoare de autoinducție (**Back EMF**) generate la comutația PWM.

2. Sistemul de Iluminat (LED):

- Utilizează un tranzistor NPN **S8050** (I_c max 700mA, Fig. 2.4).
- Permite controlul intensității luminoase fără a depăși limita de curent a pinilor ESP32.

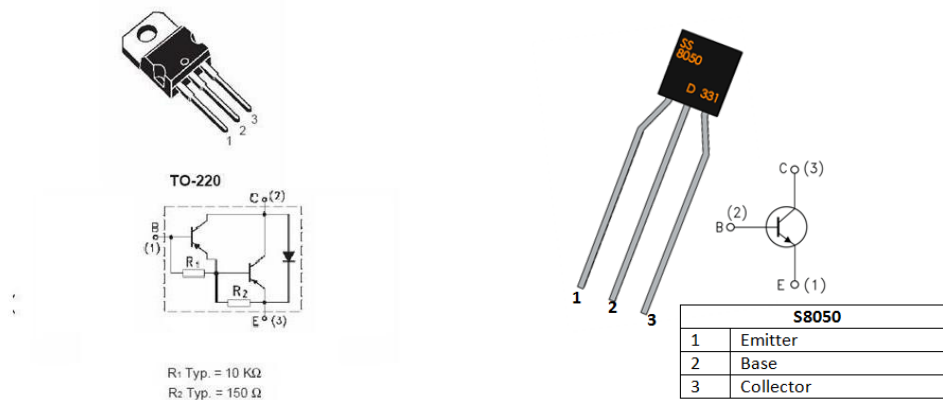


Fig. 2.4: Tranzistori TIP142 si S8050

2.4. Interfețe de Afișare

Sistemul dispune de două ecrane **LCD 16x2 cu module I2C**, care oferă feedback vizual direct (Fig 2.5) :

- Afișarea telemetriei de mediu (BME680).
- Monitorizarea parametrilor electrici (V, mA, W) și a treptei PWM active.

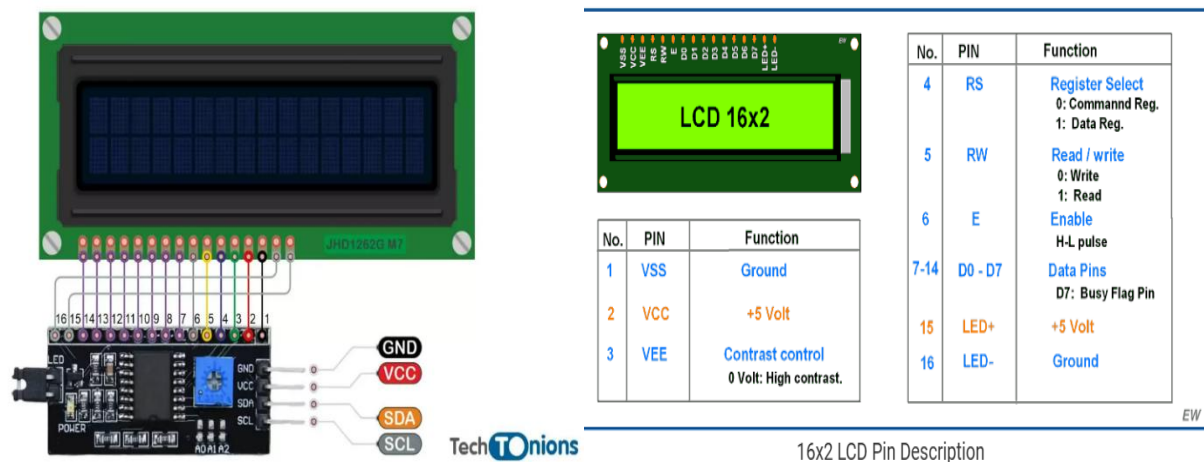


Fig. 2.5: Ecran LCD 16x2 LCD

2.5. Arhitectura si Conexiuni Hardware

Aceasta secțiune descrie modul în care componentele individuale interacționează pentru a forma un sistem unitar.

2.5.1. Diagrama Bloc a Sistemului

Diagrama bloc prezintă organizarea logică a componentelor. Aceasta pune accent pe distribuția senzorilor și a elementelor de execuție, fără a detalia nivelurile de tensiune, evidențiind rolul central al brokerului MQTT în comunicarea dintre cele două unități ESP32. (Fig. 2.6)

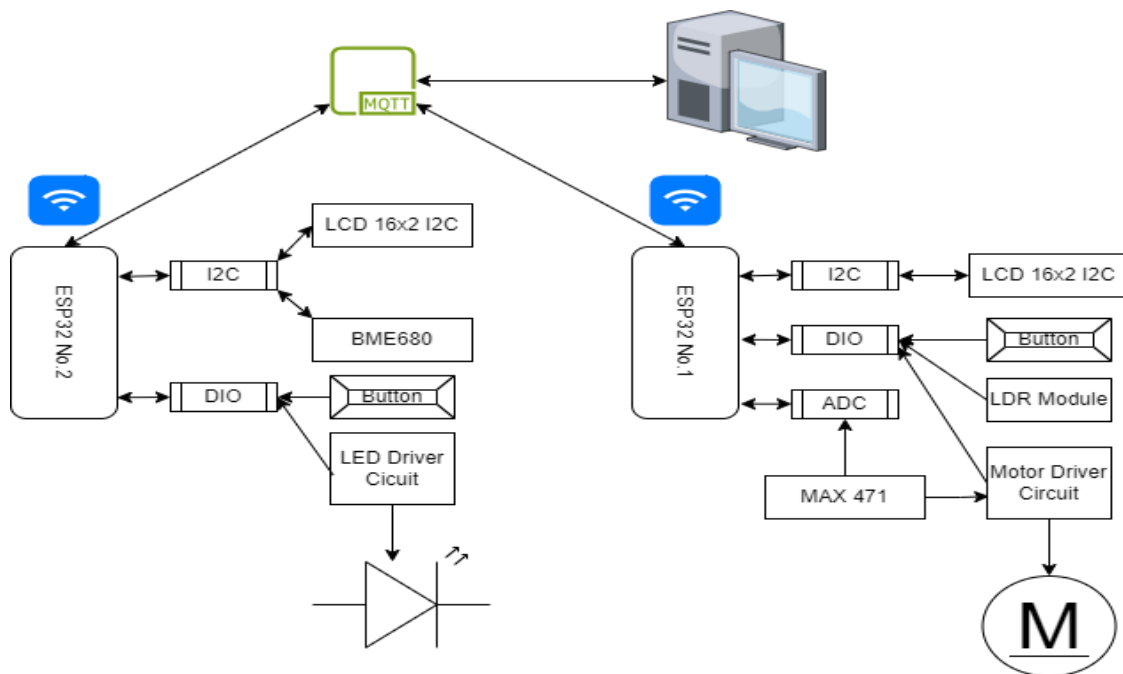


Fig. 2.6: Diagrama Bloc a Sistemului

2.5.2. Layout-ul General (Breadboard)

Implementarea fizica pe breadboard prezinta implementarea fizica a proiectului. (Fig. 2.7)

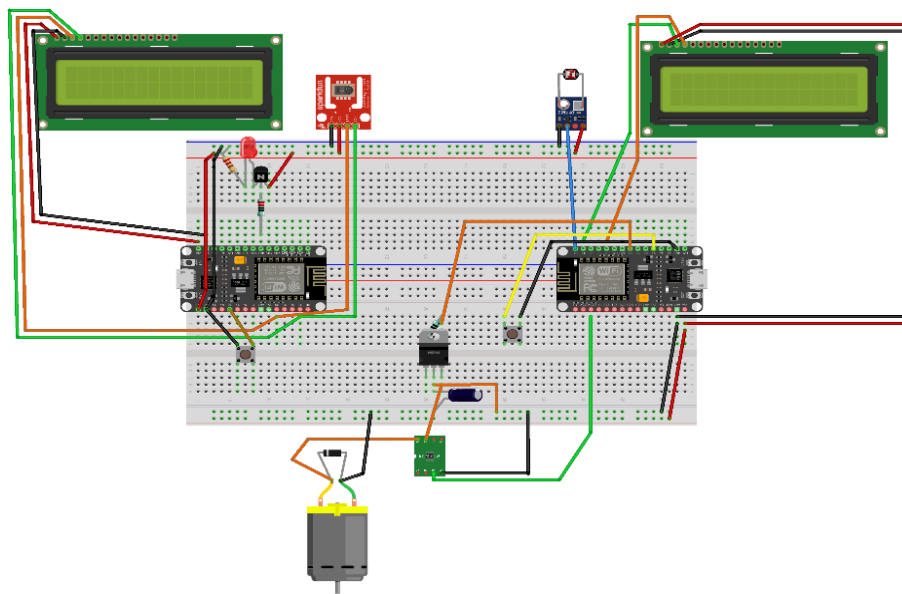


Fig. 2.7: Implementarea fizica pe breadboard

fritzing

2.5.3. Circuitele de Comanda (Drivere Actuatori)

Sistemul utilizeaza doua circuite de comanda dedicate pentru a izola microcontrolerul de sarcinile electrice mari: (Fig. 2.8)

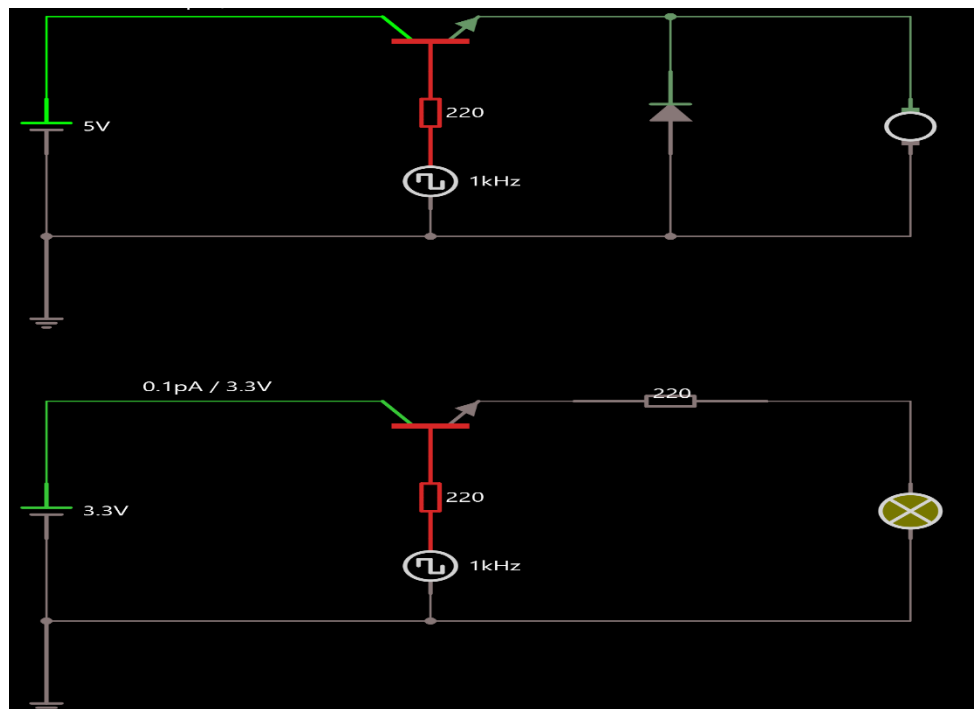


Fig. 2.8: Implementarea fizica pe breadboard

A. Circuitul de Comanda Motor (TIP142):

- **Tranzistor:** Darlington NPN TIP142.
- **Functie:** Controlul vitezei prin PWM (1 kHz).
- **Protectie:** Include o dioda Flyback montata in paralel cu motorul pentru a absorbi varfurile de tensiune generate de inductanta motorului (Back EMF).
- **Monitorizare:** Senzorul MAX471 este inseriat in acest circuit pentru a masura curentul consumat in timp real.

B. Circuitul de Comanda LED (S8050):

- **Tranzistor:** NPN S8050 de mica putere.
- **Functie:** Comutarea sau reglarea intensitatii LED-ului pe baza datelor primite de la senzorul LDR.
- **Avantaj:** Protejeaza pinul GPIO al ESP32, permitand un curent de operare mai mare decat cei 12-20mA pe care ii poate furniza direct microcontrolerul.

Nota Tehnica: Cuplul nominal al motorului este de 10 g*cm (aprox. 0.139 oz*in). In cadrul testelor, s-a observat ca la valori PWM sub 45%, forta motorului este limitata de frecarea interna si de caderea de tensiune de aprox. 1.2V pe tranzistorul TIP142.

Capitolul 3: Arhitectura Software

Sistemul utilizeaza o arhitectura modulara. Fiecare componenta hardware este gestionata de un modul software dedicat, facilitand testarea si depanarea independenta.

3.1. Module Specifice Nod 1 (Figura 3.1)

- **Setup & Loop:** Initializeaza perifericele(`Serial.begin(115200)`, `Max471_Init(MAX_471_PIN, ADC_RESOLUTION, ADC_0db, LdrHdl_Init())`, `MotorCtrl_Init()`, `WifiMng_LoadNetworksInVector()`, `TimeMng_Init()`, `LcdMng_I`

`nit()`, `MqttMng_Init()`) si ruleaza secvential functiile de control la un interval de 20ms pentru a reseta Watchdog timer-ul si a nu bloca procesorul.

- **Motor Control** (`MotorCtrl_MainFunction()`): Verifica starea conexiunii MQTT si modul de operare (`Manual[Overdrive:True]/Auto[Overdrive:False]`). Calculeaza turatia aproximativa (`MotorCtrl_ProcessRPM()`), starea curenta a motorului (`MotorCtrl_CheckMotorStat(targetDuty)`) si gestioneaza pornirea/oprirea motorului.
- **Max471_Hdl()**: Realizeaza achizitia de date prin oversampling (citiri multiple pentru precizie), converteste valorile analogice si aplica un filtru de medie mobila (moving average filter) pentru a elimina zgomotul electric cauzat de motor.
- **LdrHdl_MainFunction()**: Monitorizeaza intrarea digitala a senzorului de lumina pentru a determina pragul de zi/noapte.

3.2. Module Specifice Nod 2 (Figura 3.2)

- **Setup & Loop**: Pe langa initializarile standard, acest nod creeaza un Task dedicat pentru BME680 (`xTaskCreatePinnedToCore`), profitand de arhitectura dual-core a ESP32 pentru a nu bloca bucla principala in timpul citirilor lungi ale senzorului.
- **LedCtrl_MainFunction()**: Gestioneaza starea LED-ului. Verifica starea conexiunii MQTT si modul de operare (`Manual[Overdrive:True]/Auto[Overdrive:False]`). In modul "Auto", foloseste un sistem de tip Cyclic Dimmer (`LedCtrl_CyclicDimmer()`) pentru a regla intensitatea luminii in mod progresiv.
- **BME680_TaskFunction(void * pvParameters) & BME680_MainFunction()**: Citeste datele brute de la senzor, calculeaza indicii de confort (ITU) si calitatea aerului (IAQ), apoi stocheaza valorile pentru a fi publicate.

3.3. Module Comune (Partajate intre Noduri)

Aceste module asigura infrastructura de comunicatie si interfata cu utilizatorul pe ambele dispozitive:

- **WifiMng_MainFunction()**: Gestioneaza scanarea si conectarea si reconectarea automata la retea. Include logica de verificare a timpului prin NTP (Network Time Protocol, `TimeMng_GetTimestamp()`) pentru a sincroniza evenimentele.
- **MqttMng_MainFunction()**: Subscrie la topicurile de comanda si publica telemetria (datele senzorilor) catre broker. Gestioneaza reconectarea automata in caz de pierdere a semnalului.
- **BtnMng_MainFunction(BTN_PIN, BtnMng_debounceTime)**: Implementeaza o masina de stare pentru tratarea apasarii (Released, Press Pending, Pressed, Release Pending), eliminand zgomotul mecanic prin debounce.
- **LcdMng_MainFunction()**: Controleaza afisajul. Permite navigarea intre mai multe ecrane de date la apasarea butonului si actualizeaza informatiile in timp real la intervale setate.

3.4. Debugger Service

Fiecare nod ruleaza un serviciu de monitorizare (`DebuggerService()`) care trimite catre consola Seriala un raport detaliat despre starea interna a sistemului, facilitand diagnosticarea rapida fara a fi nevoie de acces fizic la ecranele LCD.

3.5. Optimizari si Tehnici de Control Avansate

Pentru a asigura stabilitatea sistemului si protectia hardware-ului, au fost implementate urmatoarele solutii software:

3.5.1. Filtrarea Datelor: Moving Average Filter si ADC Vref 1.1V

Senzorul MAX471 este montat in proximitatea motorului, fiind expus la zgomot electromagnetic. Pentru a obtine o citire stabila a curentului si a tensiunii:

- S-a utilizat tehnica oversampling-ului, citind senzorul de mai multe ori intr-un interval scurt.
- Datele sunt procesate printr-un filtru de medie mobila (Moving Average), care elimina varfurile (spike-urile) cauzate de comutatia PWM si de Back EMF, oferind o valoare "curata" pe LCD si in Dashboard.
- Tensiunea de referinta a ADC-ului este setata la 1.1V (ADC_0db) deoarece motorul folosit nu va trage mai mult de 1A, si legea de conversie fiind 1A la 1V, ajuta la o masurare mai optima.

3.5.2. Managementul PWM (1 kHz):

Frecventa semnalului PWM pentru controlul motorului a fost setata la 1 kHz cu o rezolutie de 8 biti:

- De ce 1 kHz? Aceasta frecventa este suficient de mare pentru a asigura o rotatie fluida si pentru a evita tiuitul audibil al motorului, dar suficient de mica pentru a reduce pierderile prin incalzire pe tranzistorul TIP142.
- Controlul LED-ului de pe al doilea nod utilizeaza tot PWM prin functia `analogWrite()`, permitand tranzitii line de luminozitate (dimming) gestionate de tranzistorul S8050.

3.5.3. Functia de Kickstart: `MotorCtrl_Kickstart(targetDuty)` ;

Motoarele DC mici au o frecare statica ce poate impiedica pornirea la valori mici de PWM (ex. 45%).

- Logica de control include un mecanism de Kickstart: la fiecare pornire din starea "Stop", motorul primeste un impuls de la 100% PWM pana la turatia setata pentru o durata foarte scurta (aprox. 100-200ms).
- Acest impuls ofera energia necesara pentru a invinge inertia, dupa care sistemul revine imediat la pragul tinta (ex. 45% sau 55%).

3.5.4. Executie Non-Blocking

O caracteristica esentiala a codului este faptul ca toate functiile sunt non-blocking:

- Nu s-a utilizat functia `delay()` in interiorul modulelor (cu exceptia micii pauze de 20ms din loop pentru stabilitatea generala).
- Gestionarea timpului pentru citirea senzorilor (BME680, MAX471) sau trimiterea datelor prin MQTT se face prin compararea valorilor `millis()`.
- Acest lucru permite ESP32-ului sa raspunda instantaneu la apasarea butoanelor sau la mesajele primite de la server, chiar daca in acelasi timp proceseaza datele de mediu sau calculeaza media mobila.

LEGEND:

Arduino Function:
Function:

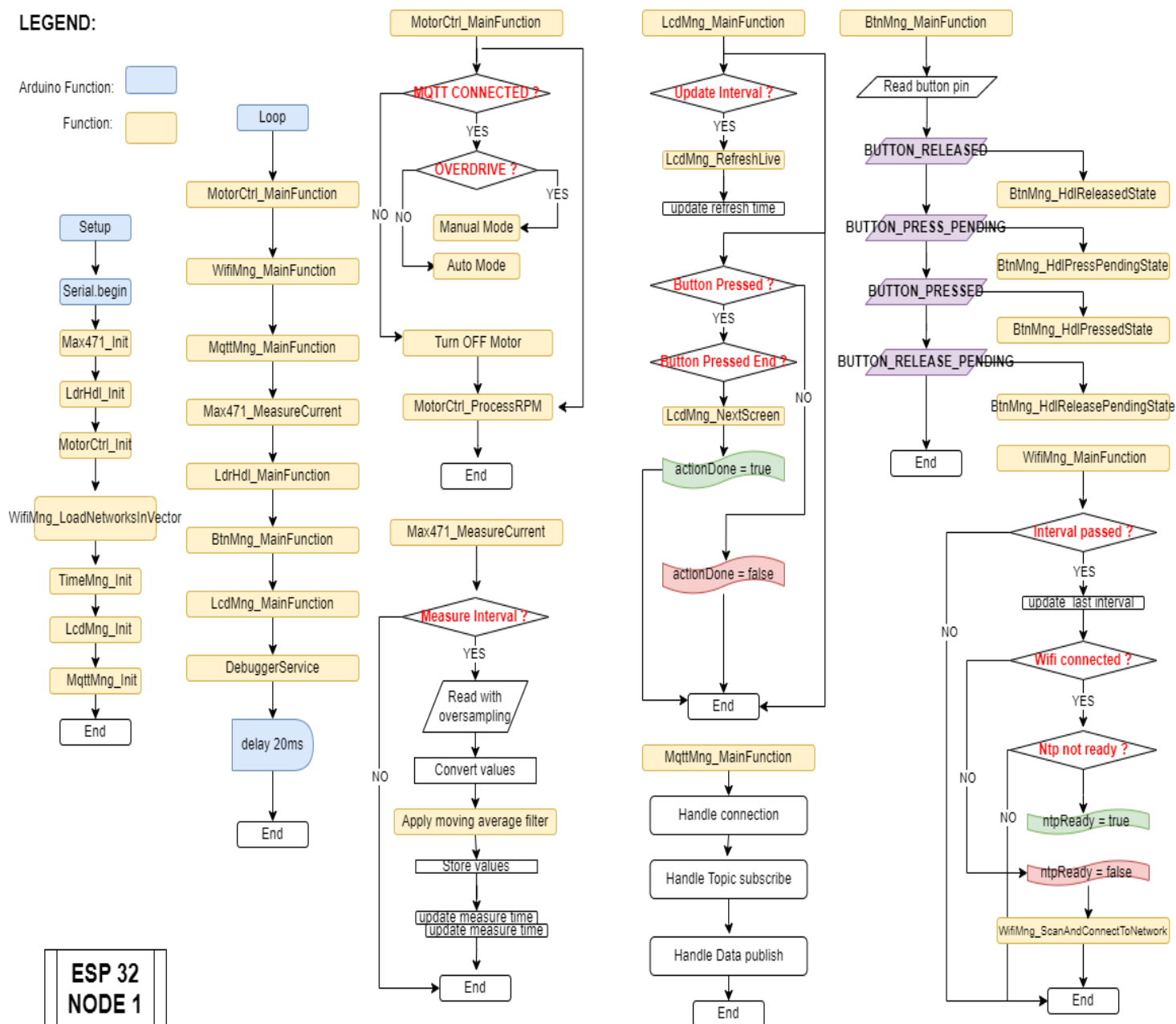


Fig 3.1: Aplicatie ESP32 Nod 1

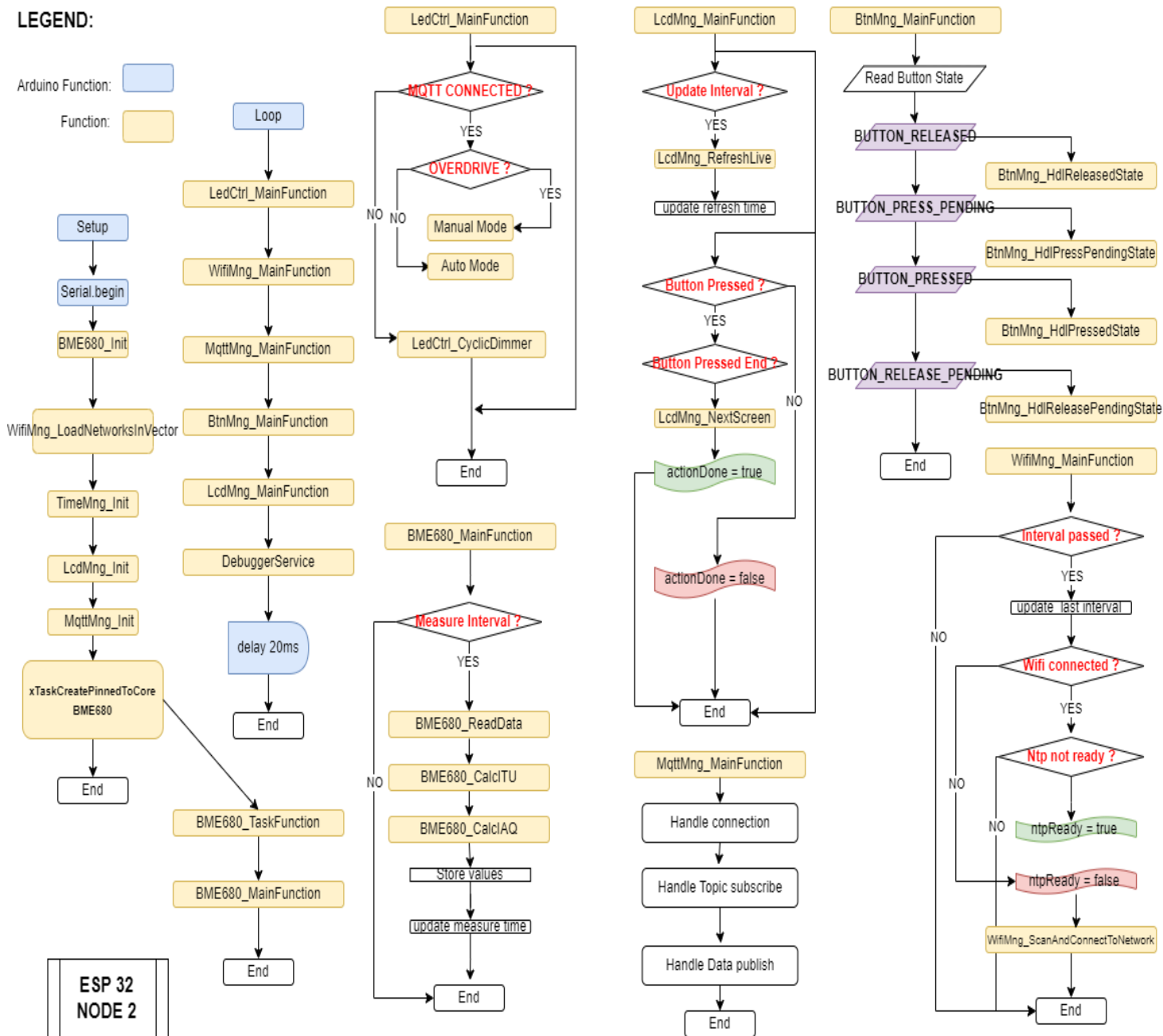


Fig 3.2: Aplicatie ESP32 Nod 2

Capitolul 4: Interfata de Control (Node-RED si MQTT)

Aceasta sectiune descrie infrastructura software de nivel inalt care asigura comunicarea, stocarea datelor si interfata grafica pentru utilizator (Dashboard).

4.1. Serverul MQTT (Mosquitto)

Serverul MQTT actioneaza ca un "broker" (intermediar) de mesaje. Acesta primeste datele de la cele doua ESP32 si le redirectioneaza catre Node-RED.

- **Instalare si Rulare:** S-a utilizat **Eclipse Mosquitto**. Instalarea pe sistemul gazda (ex: Raspberry Pi sau PC) s-a realizat prin managerul de pachete, fiind configurat sa ruleze ca serviciu de fundal pe portul standard 1883, logare tip "anonymous" pentru a nu fi nevoie de user parola si a facilita protiparea
- **Rol:** Asigura decuplarea completa intre noduri; ESP1 nu are nevoie sa stie adresa IP a ESP2, ambele comunicand doar cu adresa serverului MQTT. (Fig. 4.1)

```
Microsoft Windows [Version 10.0.26280.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dan>cd "C:\Program Files\mosquitto"

C:\Program Files\mosquitto>mosquitto.exe -v -c mosquitto.conf
1769769556: mosquitto version 2.0.22 starting
1769769556: Config loaded from mosquitto.conf.
1769769556: Opening ipv4 listen socket on port 1883.
1769769556: Opening ipv6 listen socket on port 1883.
1769769556: mosquitto version 2.0.22 running

1769769556: New connection from 192.168.0.10:58884 on port 1883.
1769769556: New client connected from 192.168.0.10:58884 as nodered7b613d974fcb7a10 (p2, c1, k60).
1769769556: No will message specified
1769769556: Sending CONNACK to nodered7b613d974fcb7a10 (0, 0)
1769769556: Received SUBSCRIBE from nodered7b613d974fcb7a10
1769769556: casa/nod2/data (QoS 2)
1769769556: nodered7b613d974fcb7a10 2 casa/nod2/data
1769769556: casa/nod1/date (QoS 2)
1769769556: nodered7b613d974fcb7a10 2 casa/nod1/date
1769769556: casa/nod1/overdrive (QoS 2)
1769769556: nodered7b613d974fcb7a10 2 casa/nod1/overdrive
1769769556: casa/nod2/overdrive (QoS 2)
1769769556: nodered7b613d974fcb7a10 2 casa/nod2/overdrive
1769769556: Sending SUBACK to nodered7b613d974fcb7a10

1769769559: New connection from 192.168.0.52:55804 on port 1883.
1769769559: New client connected from 192.168.0.52:55804 as ESP32_Node_2 (p2, c1, k15, u'user').
1769769559: Will message specified (5 bytes) (r1, q1).
1769769559: casa/nod2/overdrive
1769769559: Sending CONNACK to ESP32_Node_2 (0, 0)
1769769559: Received SUBSCRIBE from ESP32_Node_2
1769769559: casa/nod1/LDR (QoS 0)
1769769559: ESP32_Node_2 0 casa/nod1/LDR
1769769559: Sending SUBACK to ESP32_Node_2
1769769559: Received SUBSCRIBE from ESP32_Node_2
1769769559: casa/nod2/overdrive (QoS 0)
1769769559: ESP32_Node_2 0 casa/nod2/overdrive
1769769559: Sending SUBACK to ESP32_Node_2
1769769559: Received SUBSCRIBE from ESP32_Node_2
1769769559: casa/nod2/set_pwm (QoS 0)
1769769559: ESP32_Node_2 0 casa/nod2/set_pwm
1769769559: Sending SUBACK to ESP32_Node_2

1769769559: Received PUBLISH from ESP32_Node_2 (d0, q0, r0, m0, 'casa/nod2/data', ... (66 bytes))
1769769559: Sending PUBLISH to nodered7b613d974fcb7a10 (d0, q0, r0, m0, 'casa/nod2/data', ... (66 bytes))
1769769559: Received PUBLISH from nodered7b613d974fcb7a10 (d0, q2, r0, m25805, 'casa/nod2/temp', ... (5 bytes))
1769769559: Sending PUBREC to nodered7b613d974fcb7a10 (m25805, r0)
1769769559: Received PUBREL from nodered7b613d974fcb7a10 (Mid: 25805)
1769769559: Sending PUBCOMP to nodered7b613d974fcb7a10 (m25805)
1769769559: Received PUBLISH from ESP32_Node_2 (d0, q0, r0, m0, 'casa/nod2/data', ... (66 bytes))
1769769559: Sending PUBLISH to nodered7b613d974fcb7a10 (d0, q0, r0, m0, 'casa/nod2/data', ... (66 bytes))
1769769561: Received PUBLISH from ESP32_Node_2 (d0, q0, r0, m0, 'casa/nod2/data', ... (66 bytes))
```

Fig 4.1: Functionare Server MQTT

4.2. Platforma Node-RED

Node-RED este instrumentul de programare vizuala utilizat pentru a procesa fluxurile de date si pentru a crea interfata de control (Dashboard).

- **Instalare:** Rulat in mediul Node.js, instalat via npm.
- **Logica de Flux (Flows):** Aici am integrat diagramele de noduri pentru cele doua ESP-uri.
 - **Flow ESP 1:** Noduri de intrare MQTT pentru U, I, P si RPM, conectate la functii de procesare si elemente grafice. (Fig.4.2)
 - **Flow ESP 2:** Noduri pentru preluarea telemetriei BME680 (T, RH, IAQ, ITU) si transmiterea comenzilor catre LED. (Fig.4.3)

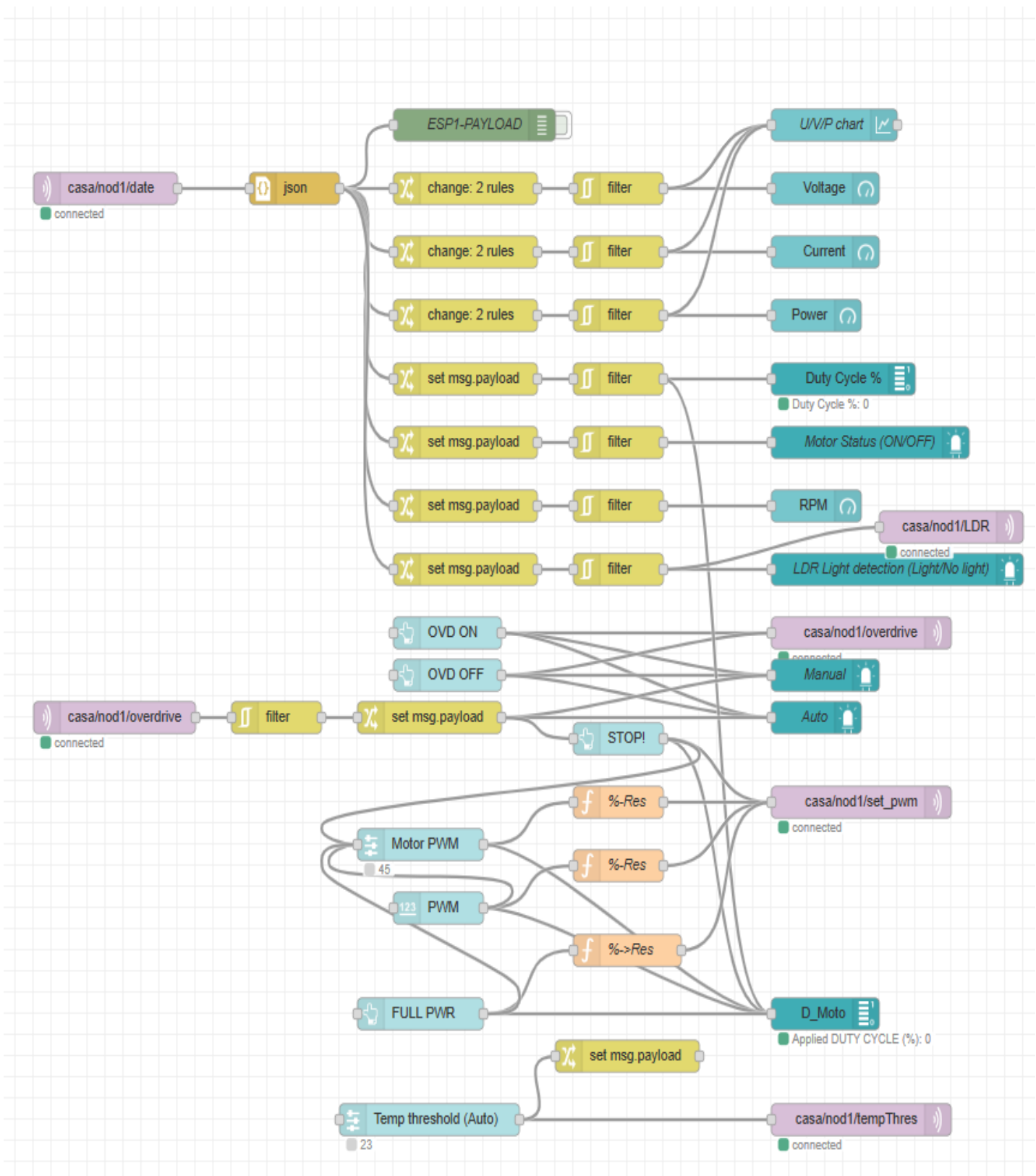


Fig 4.2: Flow ESP32 Node 1

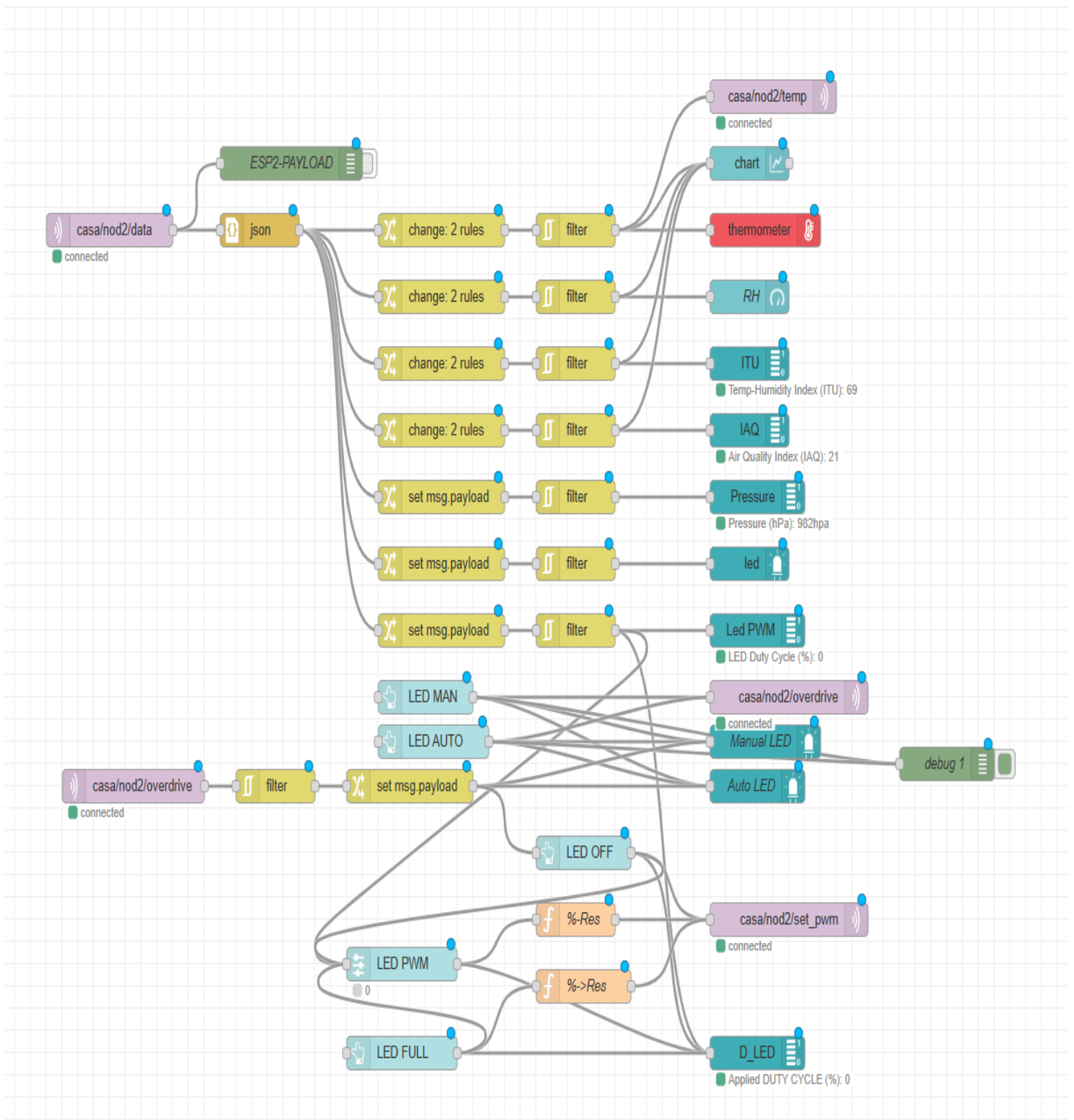


Fig 4.3: Flow ESP32 Node 2

4.3. Prezentarea Dashboard-ului (Interfata Utilizator)

Dashboard-ul este impartit in trei panouri principale pentru o monitorizare clara si un control precis.

4.3.1. Panoul 1: Monitorizare Energie si Motor (Nod 1)

Acest panou este dedicat performantei electrice a motorului DC. (Fig. 4.4)

- **Instrumente de masura:** Gauge-uri si grafice de tip "History" pentru Tensiune (U), Curent (I), Putere (P) si RPM.
- **Vizualizare Comparativa:** Chart (grafic liniar) care suprapune valorile U, I si P pentru analiza eficientei.
- **Indicatori Stare:** O bara progresiva pentru nivelul de **Duty Cycle** al motorului si un LED virtual care semnalizeaza daca motorul este alimentat.

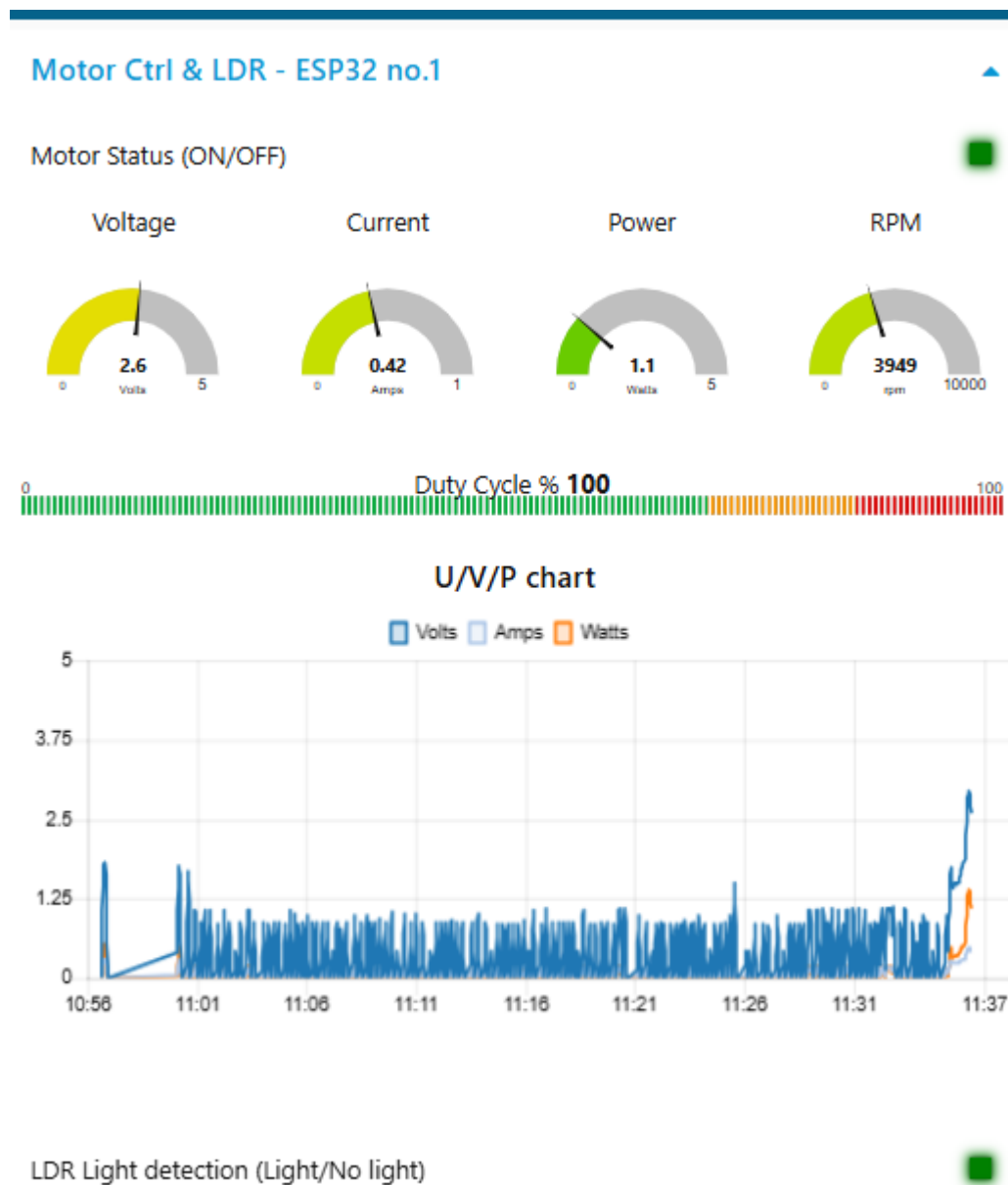


Fig 4.4: Panoul 1: Monitorizare Energie si Motor (Nod 1)

4.3.2. Panoul 2: Monitorizare Mediu si Senzori (Nod 2)

Ofera o privire de ansamblu asupra conditiilor ambientale citite de BME680 si starea luminilor. (Fig. 4.5)

- **Indicatori Vizuali:** "Fill tank" pentru Umiditate Relativa (RH), termometru vertical pentru Temperatura (T), bare de progres pentru calitatea aerului (IAQ), indicele de confort (ITU) si presiune.
- **Istoric Date:** Grafice dedicate pentru evolutia RH, T, IAQ si ITU.
- **Status Sistem:** Bara pentru Duty Cycle LED, LED-uri virtuale pentru starea Motor ON/OFF si un indicator pentru detectia de lumina (zi/noapte) primita de la LDR.

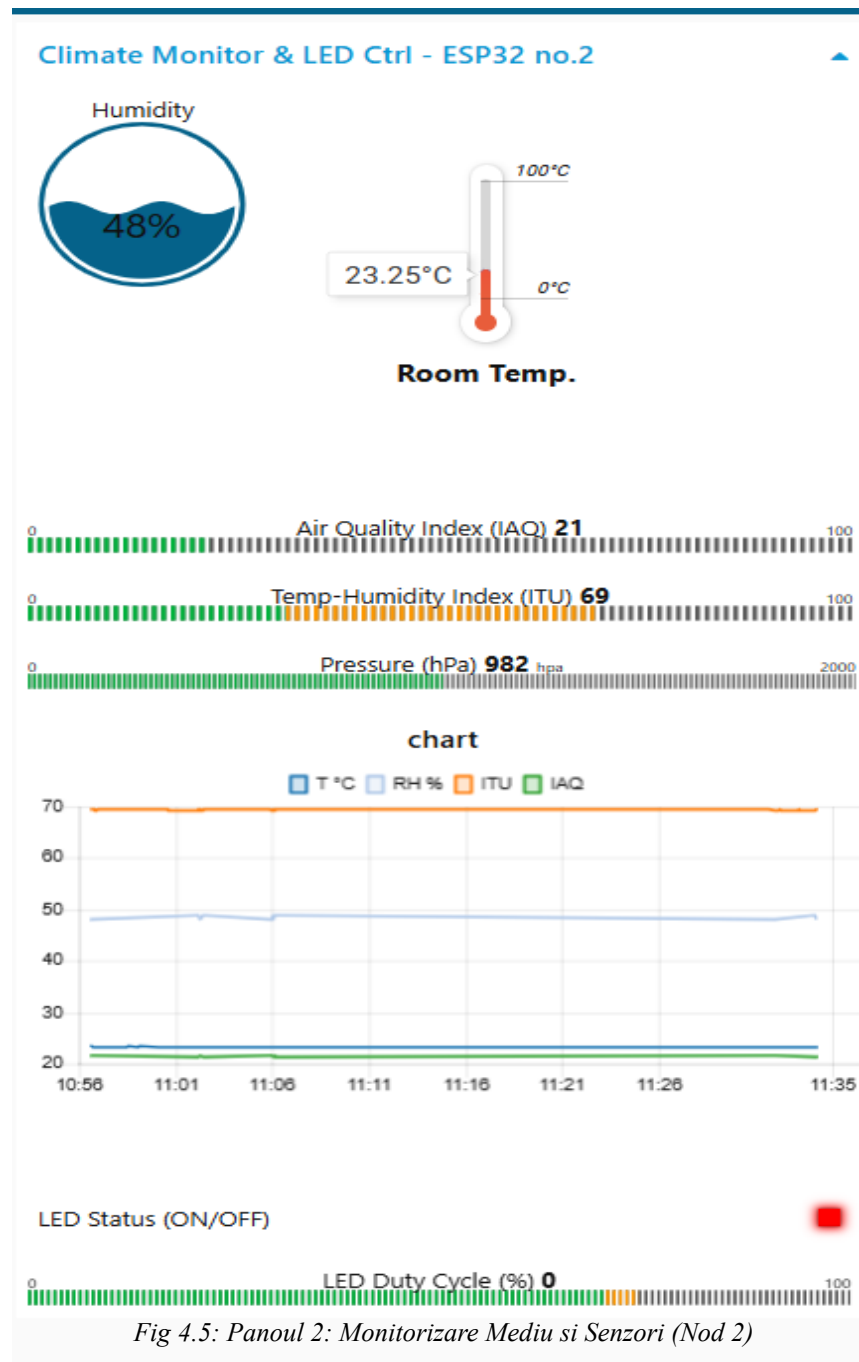


Fig 4.5: Panoul 2: Monitorizare Mediu si Senzori (Nod 2)

4.3.3. Panoul 3: Consola de Comanda (Control Motor si LED)

Acesta este panoul interactiv unde utilizatorul poate interveni asupra sistemului. (Fig.4.6)

Control Motor:

- **Moduri:** LED-uri de stare pentru modul Auto (ON/OFF) si Manual (ON/OFF).
- **Comenzi Rapide:** 4 butoane pentru Overdrive ON, Overdrive OFF, Full Power si Stop.
- **Reglaj Fin:** Control numeric pentru setarea exacta a turatiei in plaja de siguranta (45-75%).
- **Interfata Tactila:** Sliding bar pentru controlul vitezei (45-100%) si un sliding bar pentru setarea pragului de temperatura (Threshold 0-30°C) utilizat in modul Auto.

Control LED:

- **Moduri:** LED-uri pentru modul Auto/Manual.
- **Comenzi Rapide:** Butoane pentru Overdrive, Full Dimm (intensitate maxima) si OFF.
- **Reglaj:** Control numeric (45-75%) si sliding bar pentru ajustarea PWM-ului de la 0 la 100%.

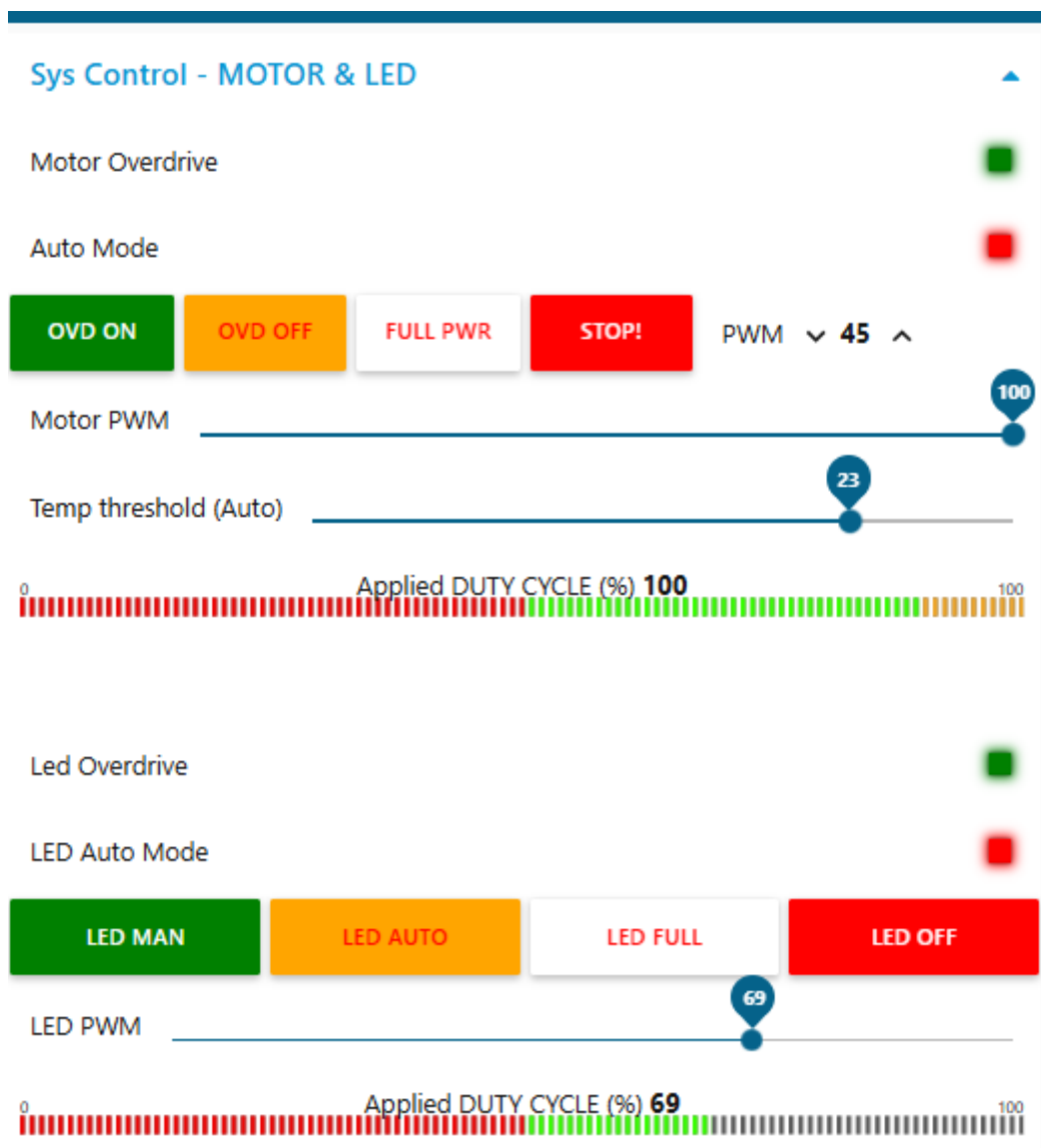


Fig 4.5: Panoul 3: Consola de Comanda (Control Motor si LED)

Capitolul 5: Concluzii

Proiectul a demonstrat succesul implementării unui sistem de monitorizare și control de tip Smart Home, utilizând o arhitectură distribuită bazată pe două unități ESP32 și protocolul de comunicație MQTT.

5.1. Realizări Proiectului

- **Integrare Hardware-Software:** S-a reușit controlul eficient al sarcinilor de putere (motor DC și LED) prin utilizarea tranzistorilor TIP142 și S8050, protejând în același timp integritatea microcontrolerelor.
- **Fiabilitatea Datelor:** Implementarea filtrului de medie mobilă (**Moving Average Filter**) și a tehnicii de **oversampling** a permis obținerea unor citiri stabile de curent și tensiune, în ciuda zgomotului electromagnetic generat de motor.
- **Control Inteligent:** Algoritmul de control cu trei praguri și funcția de **Kickstart** au asigurat o funcționare fluidă a sistemului de răcire, eliminând problemele cauzate de frecarea statică a motorului la turații mici.
- **Monitorizare Centralizată:** Interfața Node-RED oferă o experiență de utilizare intuitivă, centralizând telemetria complexă (BME680, MAX471, LDR) într-un dashboard vizual complet.

5.2. Avantajele Arhitecturii Adoptate

Utilizarea protocolului MQTT a permis o scalabilitate ridicată. Nodurile pot fi plasate în locații diferite, singura condiție fiind accesul la rețeaua WiFi, ceea ce transformă acest proiect într-o soluție viabilă pentru automatizări rezidențiale sau industriale ușoare.

Execuția de tip **non-blocking** și utilizarea **multithreading-ului** (pe ESP32 Node 2) garantează că sistemul rămâne receptiv la comenzile utilizatorului, indiferent de complexitatea calculelor de mediu efectuate în fundal.

